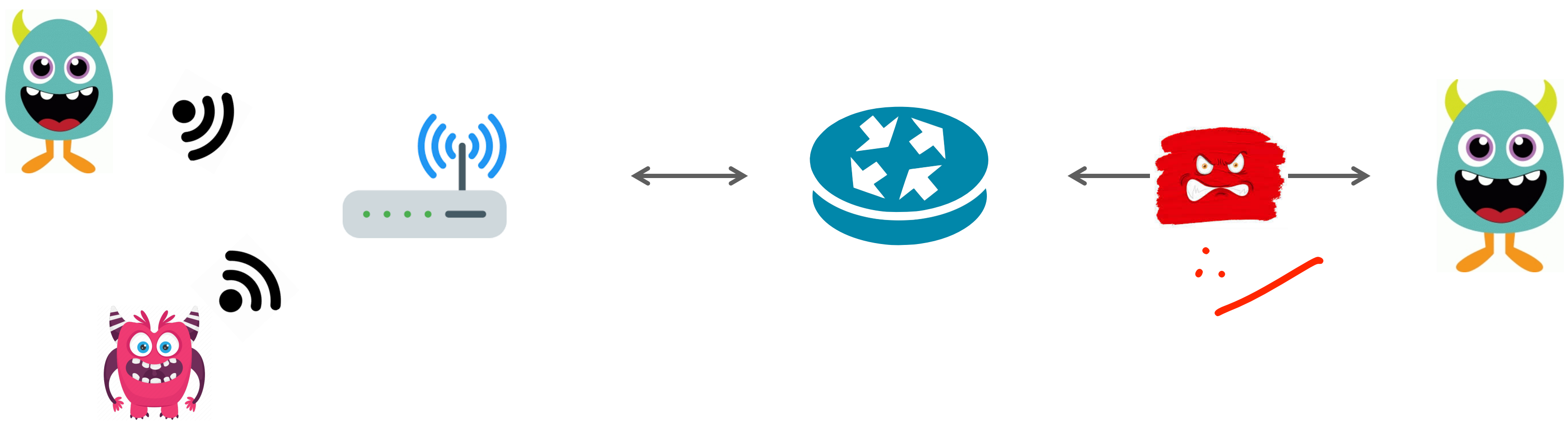
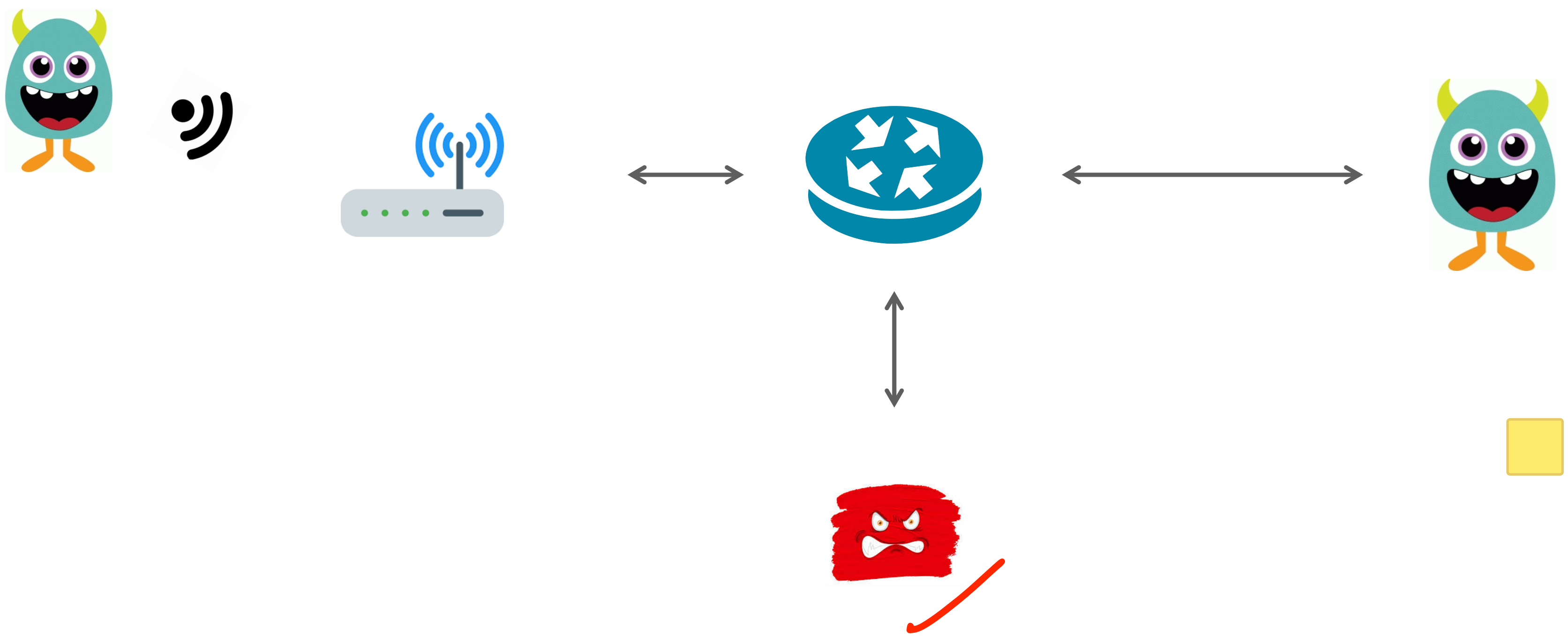
DoS Attacks and Network Defenses

Slides gathered from Computer and Network Security course, Stanford

Review: On Path Attacker



Review: Off Path Attacker



No security guarantees

Confidentiality — Ethernet, IP, UDP, and TCP do not provide any confidentiality. All traffic is in cleartext.

On-path attacker can do anything. ARP and BGP attacks allow an off-path attacker to become on-path and MITM connections.

Integrity — No guarantees that attacker hasn't modified traffic. Ethernet, IP and UDP have no protection against spoofed packets. TCP provides weak guarantee of source authentication.

Availability — Attackers can attempt to inject RST packets.

Assume network is malicious

Always Assume: The network is out to get you.

Solution: Always use TLS if you want any protection against large-scale eavesdropping (e.g., intelligence agencies), or guarantee that data hasn't been modified or corrupted by an on-path attacker

Denial of Service Attacks

Goal: take large service/network/org offline by overwhelming it with network traffic such that they can't process real requests

How: find mechanism where attacker doesn't spend a lot of effort, but requests are difficult/expensive for victim to process

Types of Attacks

DoS Bug: design flaw that allows one machine to disrupt a service. Generally a protocol asymmetry, e.g., easy to send request, difficult to create response. Or requires server state.

DoS Flood: control a large number of requests from a botnet or other machines you control

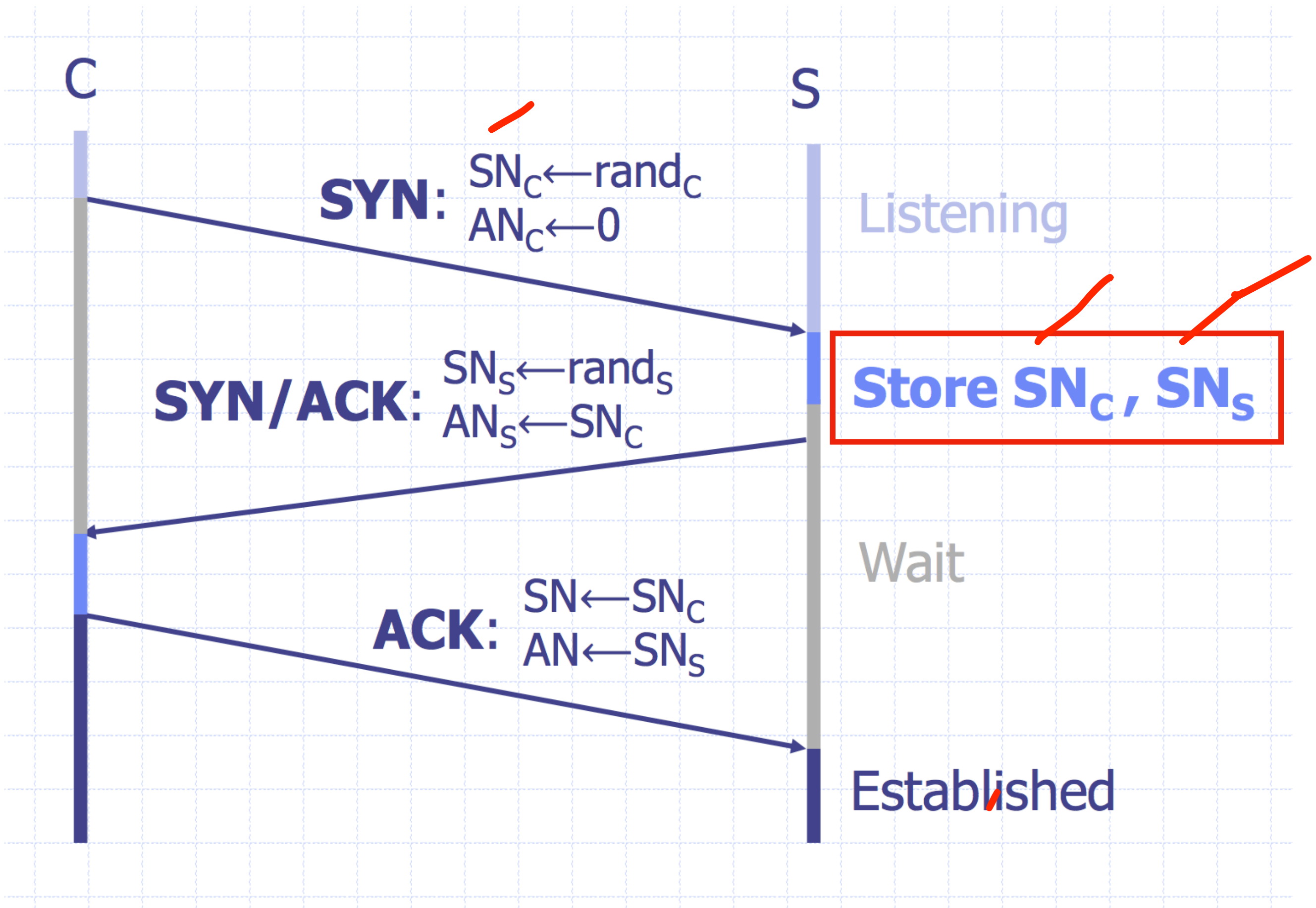
DoS Opportunities at Every Layer

L2/L3 Layer: send too much traffic for switches/routers to handle

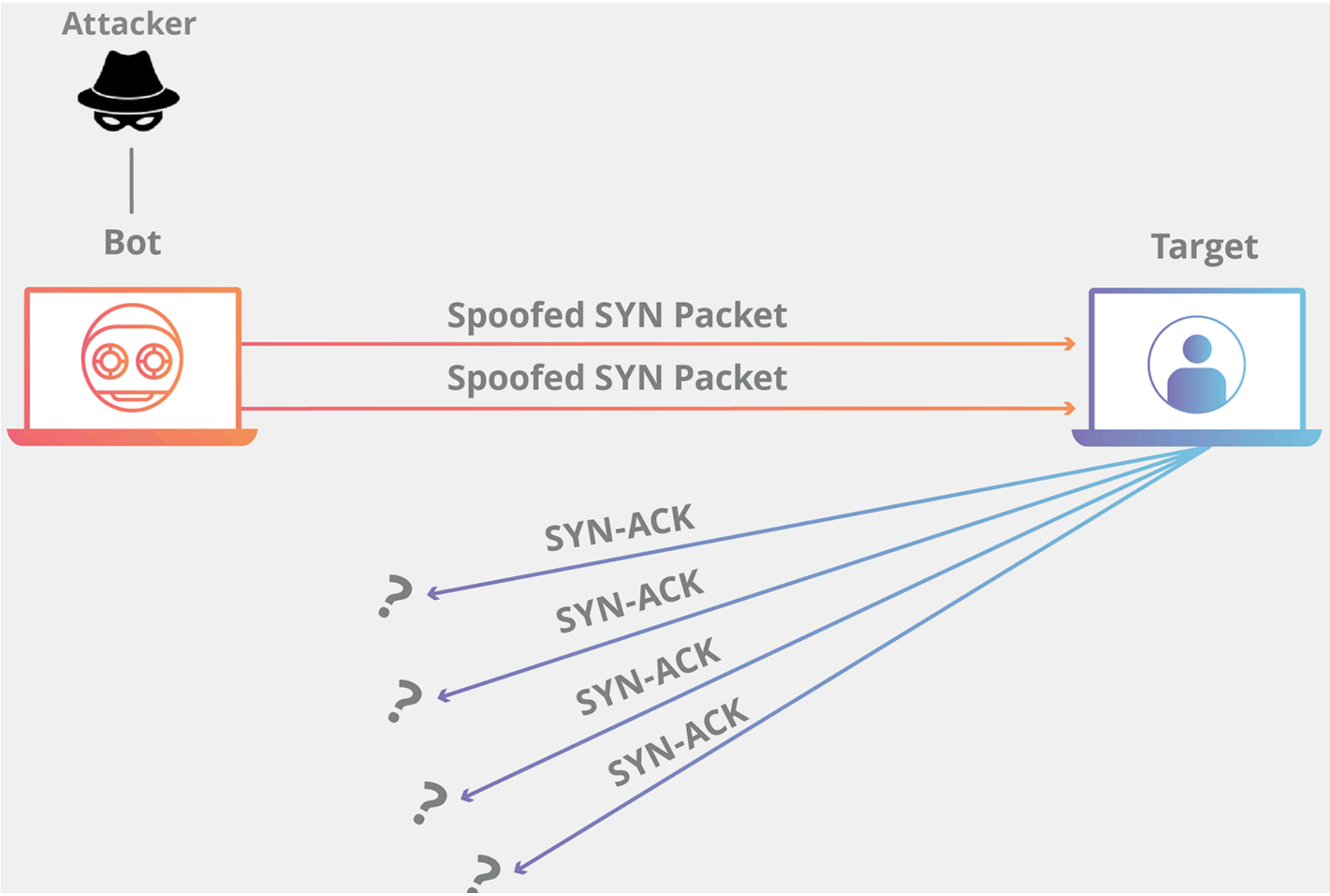
TCP/UDP: require servers to maintain large number of concurrent connections or state

Application Layer: require servers to perform expensive queries or cryptographic operations

TCP Handshake



SYN Floods


Attacker
Bot
Target
Spoofed SYN Packet
Spoofed SYN Packet
SYN-ACK
SYN-ACK
SYN-ACK
SYN-ACK

Core Problem

Problem: server commits resources (memory) before confirming identify of the client (when client responds)

Bad Solution:

- Increase backlog queue size
- Decrease timeout

Real Solution: Avoid state until 3-way handshake completes

SYN Cookies

Idea: Instead of storing SN_C and SN_S…

send a cookie back to the client.

L = MAC_{key} (SAddr, SPort, DAddr, DPort, SN_C, T)

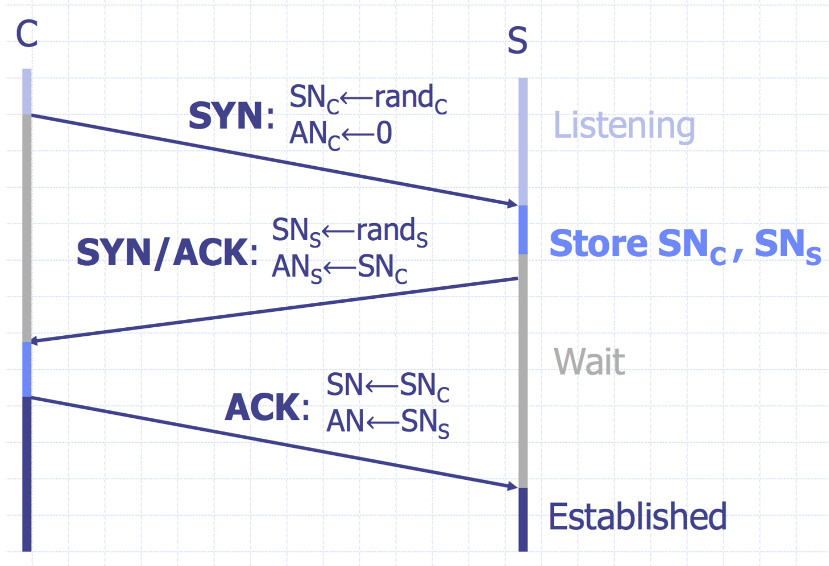
key: picked at random during boot

T = 5-bit counter incremented every 64 secs.

SN_S = (T || mss || L)

Honest client sends ACK (AN=SN_S , SN=SN_C+1)

Server allocates space for socket only if valid SN_S



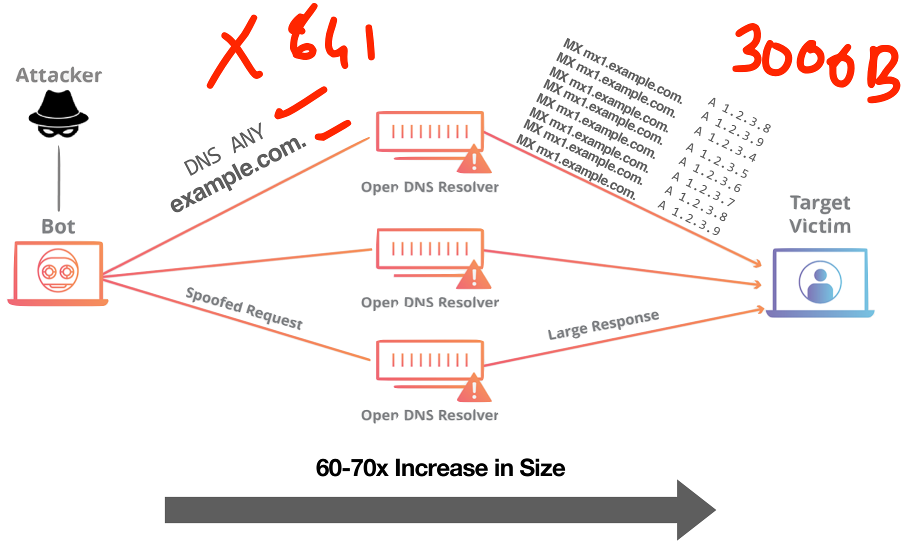
Server does not save state
(loses TCP options)

Amplification Attacks

Services that respond to a single (small) UDP packet with a large UDP packet can be used to amplify DOS attacks

Attacker forges packet and sets source IP to victim's IP address. When service responds, it sends large amount of data to the spoofed victim

The attacker needs a large number of these services to amplify packets. Otherwise, the victim could just drop the packets from the small number of hosts



Cloudfare: a recent six month window our DDoS mitigation system "Gatebot" detected 6,329 simple reflection attacks (that's one every 40 minutes)

Common UDP Amplifiers

DNS: ANY query returns *all* records server has about a domain

NTP: MONLIST returns list of last 600 clients who asked for the time recently

DNS: Do not have recursive resolvers on the public Internet.

NTP: Do not respond to commands like MONLIST

Both are considered misconfigurations today, but often 100Ks of misconfigured hosts on the public Internet

Amplification Attacks

2013: DDoS attack generated 300 Gbps (DNS)

- 31,000 misconfigured open DNS resolvers, each at 10 Mbps
- Source: 3 networks that allowed IP spoofing

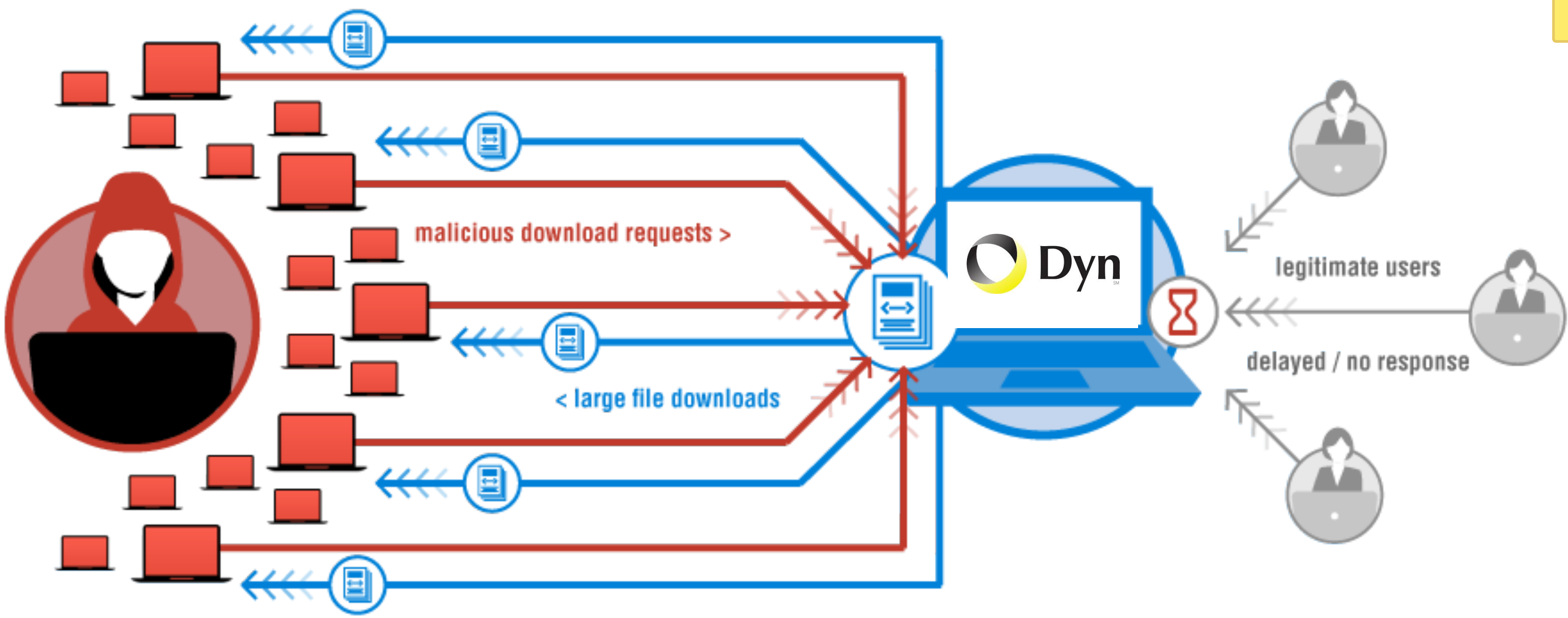
2014: 400 Gbps DDoS attacked used 4,500 NTP servers

THE WALL STREET JOURNAL.

October 21, 2016

Cyberattack Knocks Out Access to Websites

Popular sites such as Twitter, Netflix and PayPal were unreachable for part of the day



> "We are still working on analyzing the data but the estimate at the time of this report is up to 100,000 malicious endpoints. […] There have been some reports of a magnitude in the 1.2 Tbps range; at this time we are unable to verify that claim."

Image: Verisign

A Botnet of IoT Devices

200K IoT devices

Not Amplification.

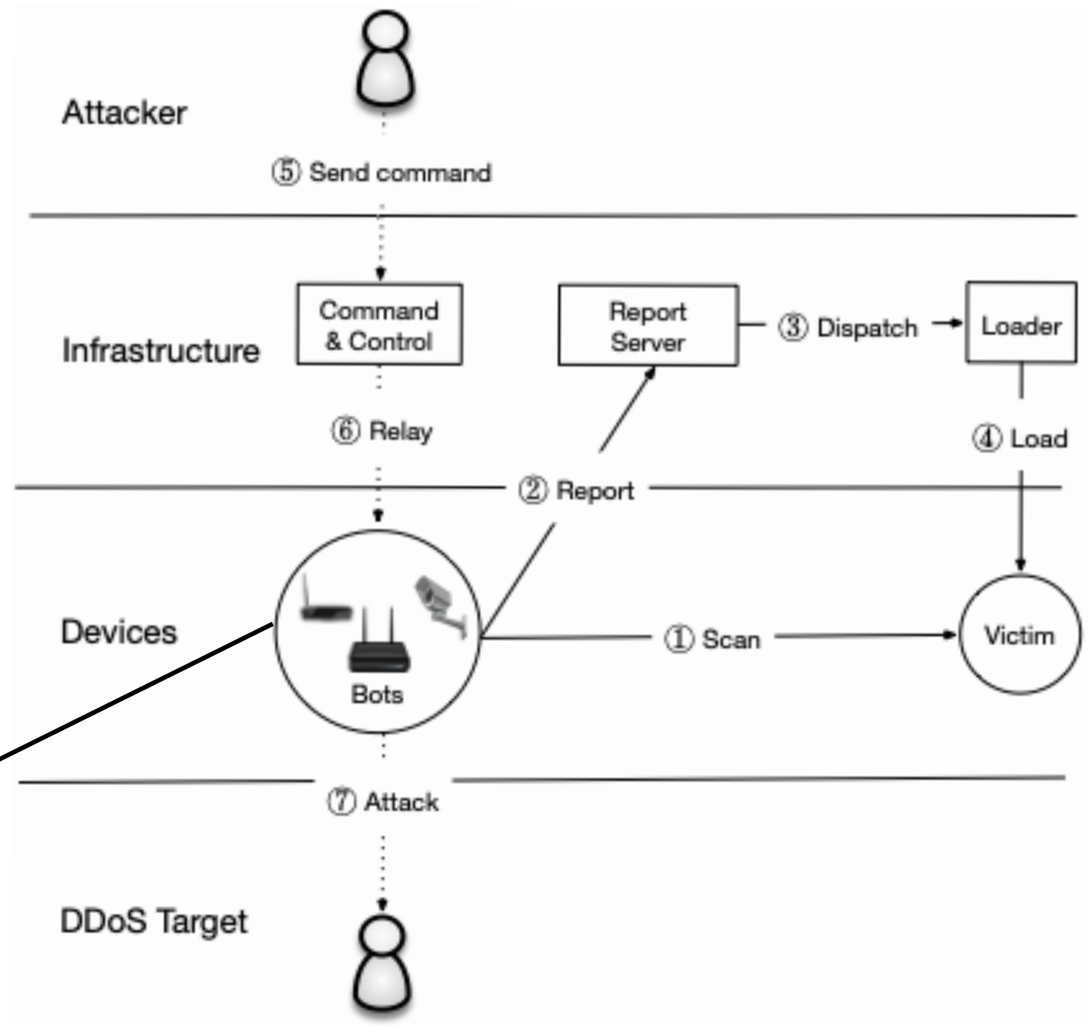
Flood with SYN, ACK, UDP, and GRE packets

The Mirai Malware

Bot master will issue commands to scan or start an attack

Attack Command:

- Action (e.g., START, STOP)
- Target IP(s)
- Attack Type (e.g., GRE, DNS, TCP)
- Attack Duration



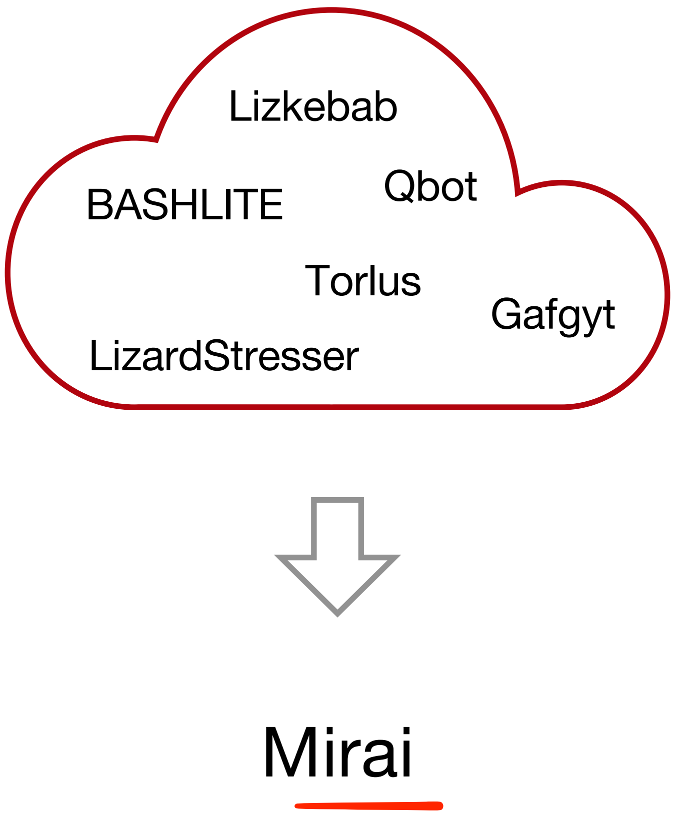
IP Cameras, routers, baby monitors, printers,

What made Mirai Successful?

The Mirai malware is (astoundingly) badly written. It uses no new or complex techniques.

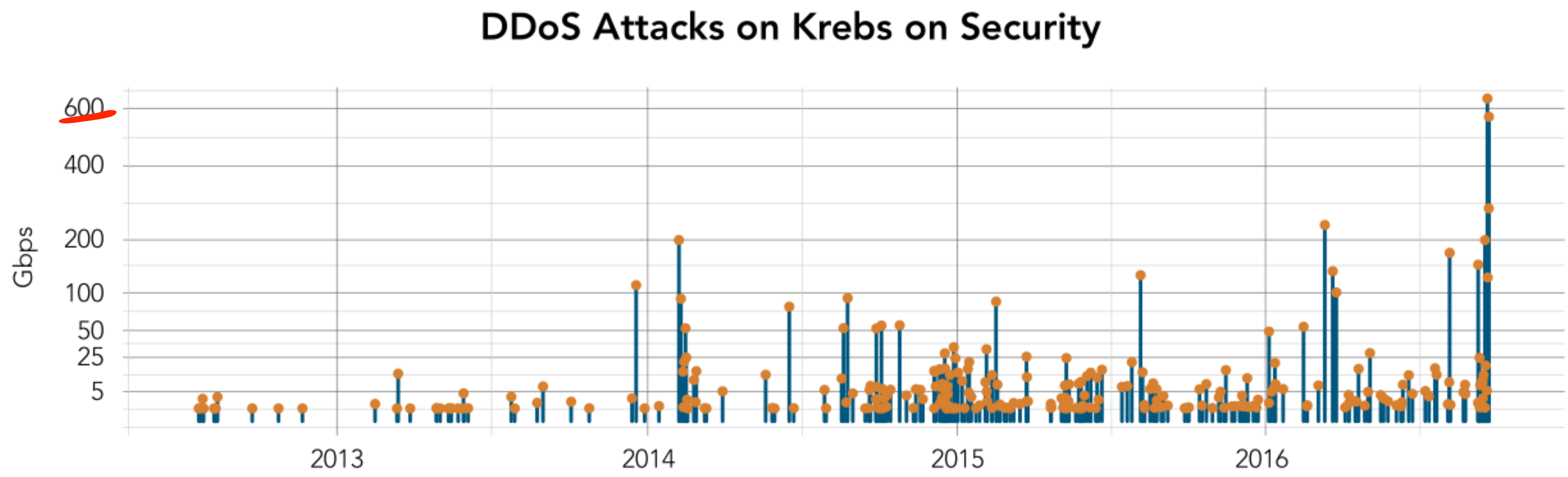
Mirai was successful because:

1. IoT security bar is very low
2. Attack simplicity enabled the malware to compromise heterogeneous hardware
3. Stateless scanning was an improvement over prior versions



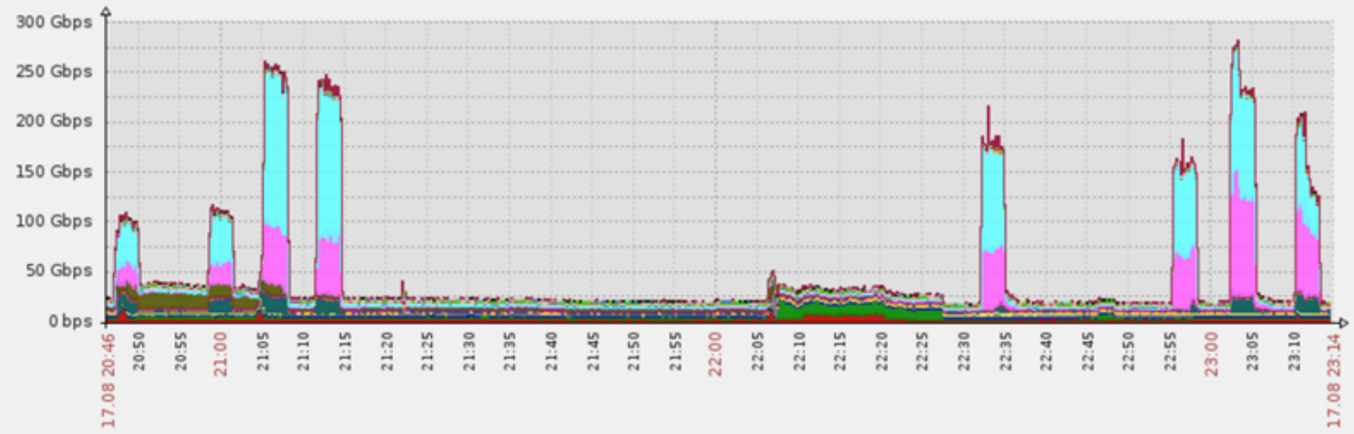
Password Guessing

Password	Device Type	Password	Device Type	Password	Device Type
123456	ACTi IP Camera	klv1234	HiSilicon IP Camera	1111	Xerox Printer
anko	ANKO Products DVR	jvbzd	HiSilicon IP Camera	Zte521	ZTE Router
pass	Axis IP Camera	admin	IPX-DDK Network Camera	1234	Unknown
888888	Dahua DVR	system	IQinVision Cameras	12345	Unknown
666666	Dahua DVR	meinsm	Mobotix Network Camera	admin1234	Unknown
vizxv	Dahua IP Camera	54321	Packet8 VOIP Phone	default	Unknown
7ujMko0vizxv	Dahua IP Camera	00000000	Panasonic Printer	fucker	Unknown
7ujMko0admin	Dahua IP Camera	realtek	RealTek Routers	guest	Unknown
666666	Dahua IP Camera	1111111	Samsung IP Camera	password	Unknown
dreambox	Dreambox TV Receiver	xmhdipc	Shenzhen Anran Camera	root	Unknown
juantech	Guangzhou Juan Optical	smcadmin	SMC Routers	service	Unknown
xc3511	H.264 Chinese DVR	ikwb	Toshiba Network Camera	support	Unknown
OxhlwSG8	HiSilicon IP Camera	ubnt	Ubiquiti AirOS Router	tech	Unknown
cat1029	HiSilicon IP Camera	supervisor	VideoIQ	user	Unknown
hi3518	HiSilicon IP Camera	<none>	Vivotek IP Camera	zlxx.	Unknown
klv123	HiSilicon IP Camera				

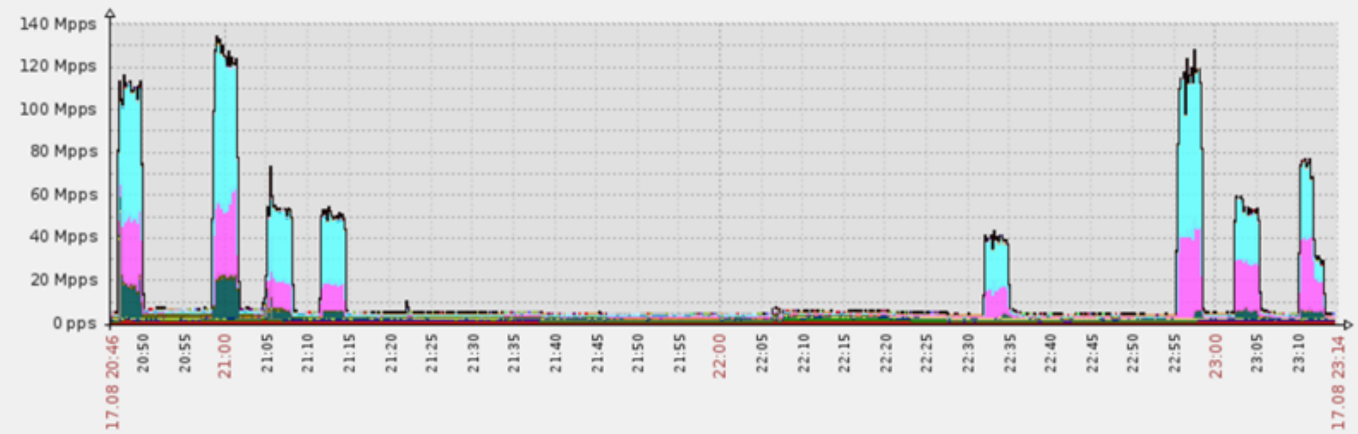


> "The magnitude of the attacks seen during the final week were significantly larger than the majority of attacks Akamai sees on a regular basis. […] In fact, while the attack on September 20 was the largest attack ever mitigated by Akamai, the attack on September 22 would have qualified for the record at any other time, peaking at 555 Gbps."

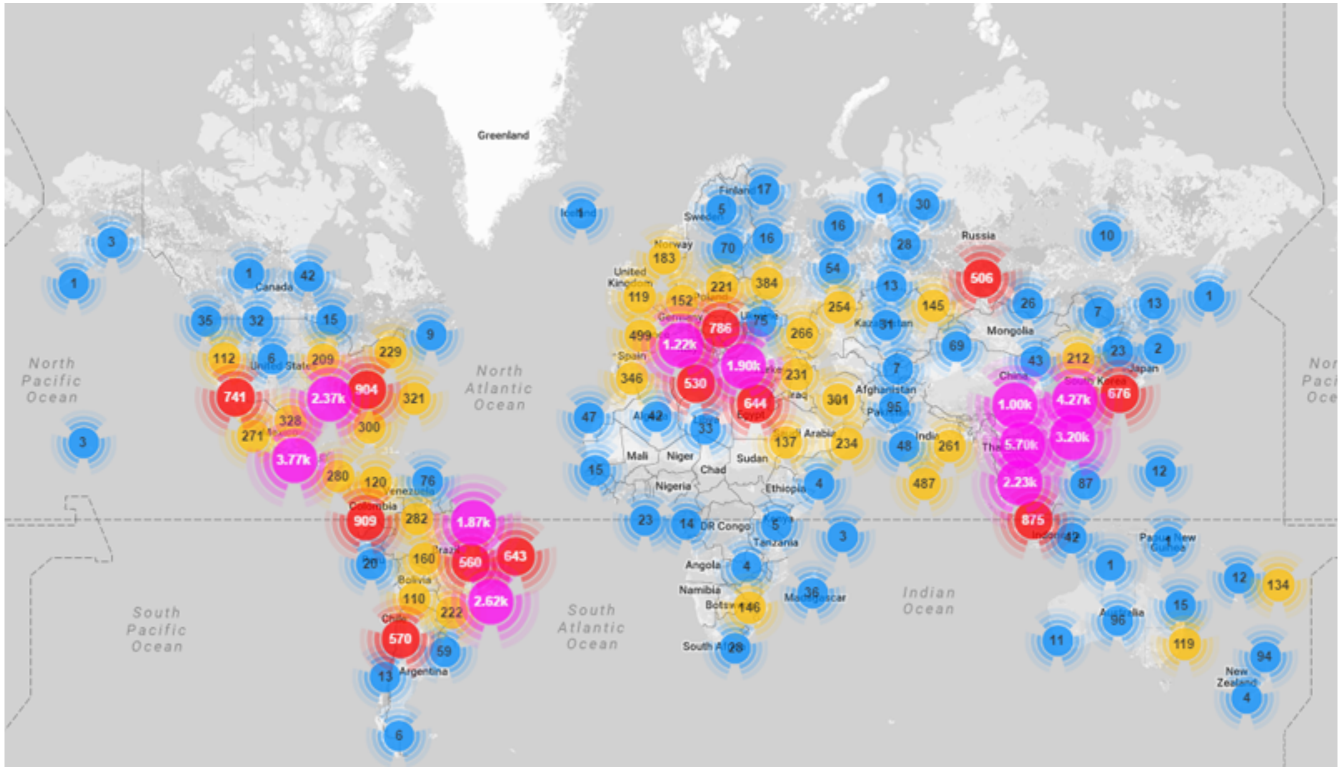
Source: 2017 Akamai State of the Internet



300 Gbps
250 Gbps
200 Gbps
150 Gbps
100 Gbps
50 Gbps
0 bps
17.08 20:46
20:50
20:55
21:00
21:05
21:10
21:15
21:20
21:25
21:30
21:35
21:40
21:45
21:50
21:55
22:00
22:05
22:10
22:15
22:20
22:25
22:30
22:35
22:40
22:45
22:50
22:55
23:00
23:05
23:10
17.08 23:14



140 Mpps
120 Mpps
100 Mpps
80 Mpps
60 Mpps
40 Mpps
20 Mpps
0 pps
17.08 20:46
20:50
20:55
21:00
21:05
21:10
21:15
21:20
21:25
21:30
21:35
21:40
21:45
21:50
21:55
22:00
22:05
22:10
22:15
22:20
22:25
22:30
22:35
22:40
22:45
22:50
22:55
23:00
23:05
23:10
17.08 23:14

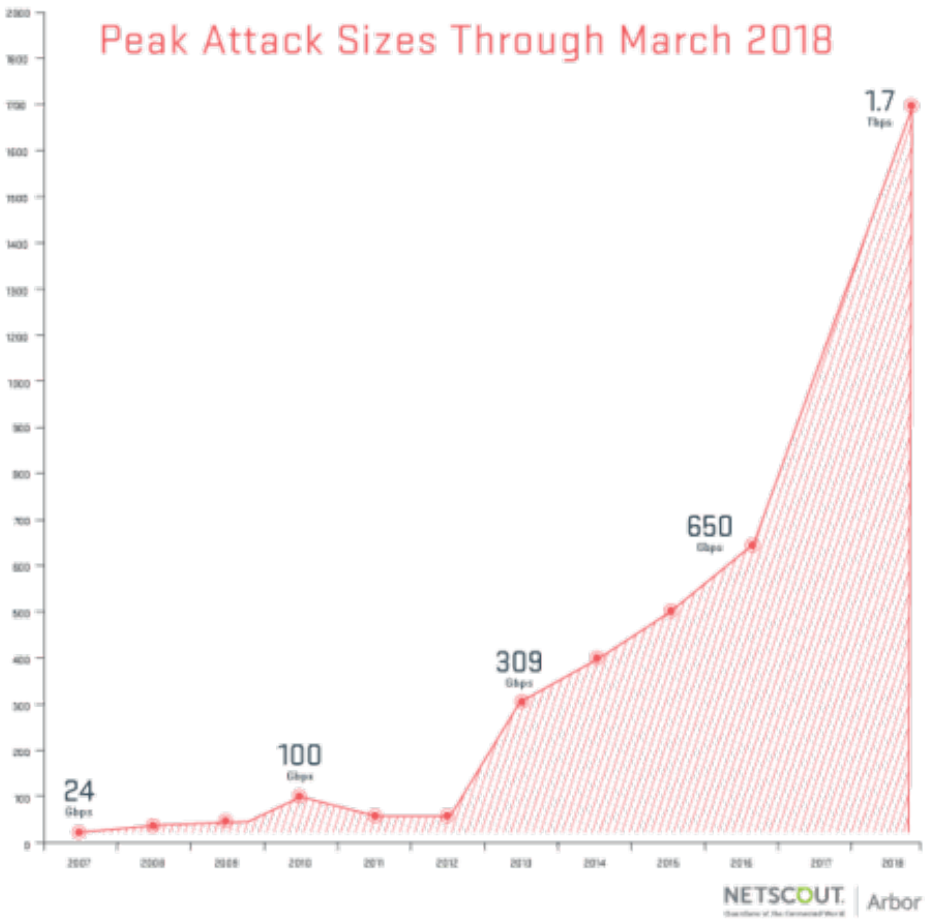


Country	% of Mirai botnet IPs
Vietnam	12.8%
Brazil	11.8%
United States	10.9%
China	8.8%
Mexico	8.4%
South Korea	6.2%
Taiwan	4.9%
Russia	4.0%
Romania	2.3%
Colombia	1.5%

Booter Services

$23.99 1 month	$34.99 1 month	$44.99 10 years
1 Month Gold	**1 Month Diamond**	**Lifetime Bronze**
Time per boot: 2400 sec	Time per boot: 3600 sec	Time per boot: 600 sec
Concurrents: 1	Concurrents: 2	Concurrents: 2
Total network: 220Gbps	Total network: 220Gbps	Total network: 220Gbps
Tools: Included	Tools: Included	Tools: Included
Support: 24/7	Support: 24/7	Support: 24/7
Buy with Paypal	Buy with Paypal	Buy with Paypal
bitcoin	bitcoin	bitcoin

Memcache



Memcache: retrieve large record

The server responds by firing back as much as 50,000 times the data it received.

Exist both a UDP and TCP version. Only works for UDP! TCP would require a three-way handshake and server would realize IP had been spoofed.

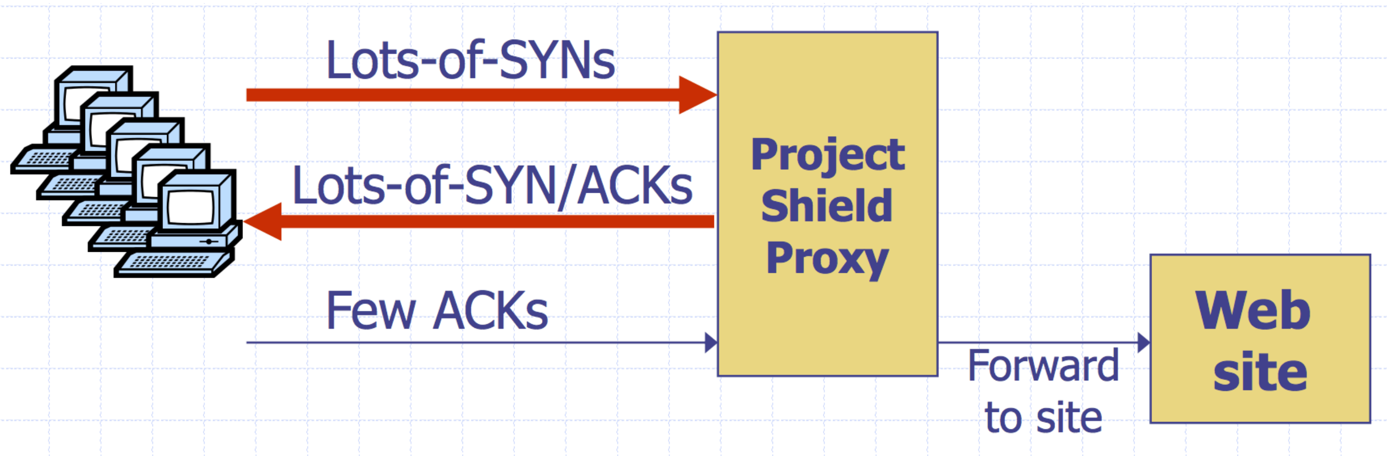
A malicious teenager calling a restaurant and saying "I'll have one of everything, please call me back and tell me my whole order." When the restaurant asks for a callback number, the number given is the targeted victim's phone number. The target then receives a call from the restaurant with a lot of information that they didn't request.

Google Project Shield

Project Shield is a free service that defends news, human rights and election monitoring sites from DDoS attacks.

DDoS Attacks are often used to censor content. In the case of Mirai, Brian Kreb's blog was under attack.

Google Project shield uses Google bandwidth to shield vulnerable websites (e.g., news, blogs, human rights orgs)



Moving Up Stack: GET Floods

Command bot army to:

* Complete real TCP connection
* Complete TLS Handshake
* GET large image or other content

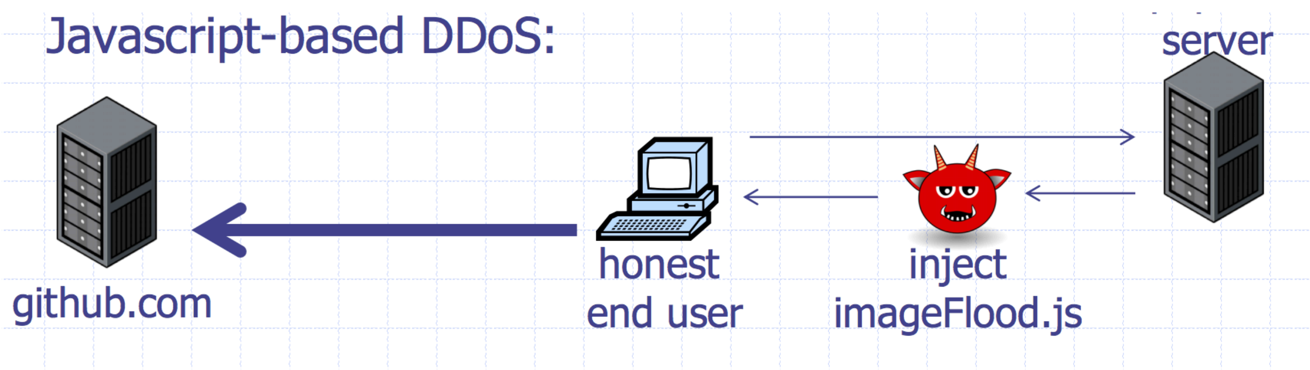
Will bypass flood protections.... but attacker can no longer use random source IPs

Victim site can block or rate limit bots

Github Attacks

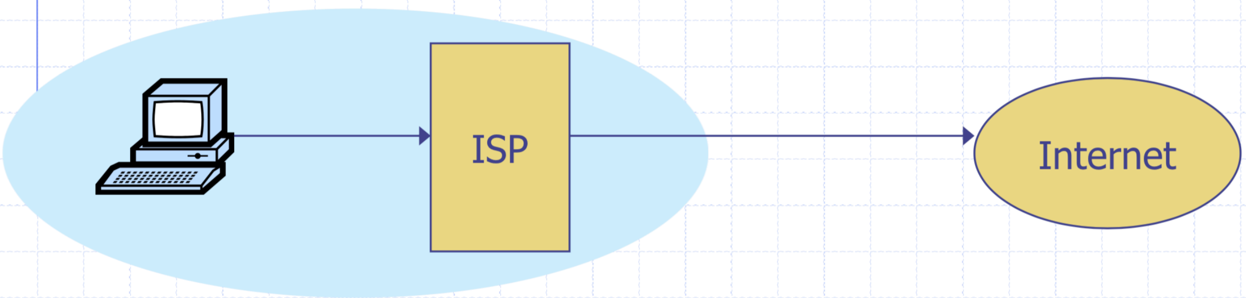
1.35 Tbps attack against Github caused by javascript injected into HTTP web requests

The Chinese government was widely suspected to be behind the attack



Ingress Filtering

- Big problem: DDoS with spoofed source IPs



- Ingress filtering policy: ISP only forwards packets with legitimate source IP (see also SAVE protocol)

Ingress Filtering

All ISPs need to do this — requires global coordination

If 10% of networks don't implement, there's no defense

No incentive for an ISP to implement — doesn't affect them

As of 2017 (from CAIDA):

33% of autonomous systems allow spoofing

23% of announced IP address space allow spoofing

2013 300 Gbps attack sent attack traffic from only 3 networks

Building a network protocol

Don't build network proto from scratch

- Never roll your own crypto
- Many opportunities to mess up parsing network packets

gRPC: http2 + TLS 1.3 RPC framework

- Safe parsing in 11 languages
- Exceptionally efficient
- Streaming/Sync/Async
- TLS-based authentication

```
syntax = "proto3";

package calc;

message AddRequest {
  int32 n1 = 1;
  int32 n2 = 2;
}

message AddReply{
  int64 res = 1;
}

service Calculator {
  rpc Add(AddRequest)           returns (AddReply) {}
  rpc Substract(SubRequest)     returns (SubReply) {}
  rpc Multiply(MultRequest)     returns (MultReply) {}
  rpc Divide(DivideRequest)     returns (DivideReply) {}
}
```